

BOOTCAMP MYHUB · GRATUIT · 5 × 2 H

# Jour 2. Programmation Python.

Livret de séance complet : ce que tu dis, ce que tu montres, les exercices, les corrigés, et le mini-projet bio.

2 heures

Débutant

En visio · 12 h GMT

Toi à l'animation

## OBJECTIF DU SOIR

Faire parler l'ordinateur en Python, puis lui faire écrire le texte de la vitrine du jour 1.

MyHub · Session 1 · Ne pas projeter les corrigés trop tôt.



## Comment utiliser ce livret

Ce PDF est ton support d'animation. Tu n'as pas besoin d'un autre cours. Les encadrés te disent quoi faire en direct.

**À DIRE**

Le propos, les analogies, les phrases clés.

**À MONTRER**

Démo écran partagé, à faire toi d'abord.

**EXERCICE**

Ils le font. Tu restes disponible.

**CORRIGÉ**

Pour toi. Tu le dévoiles après, pas avant.

**NE PAS TRAITER CE SOIR**

Fonctions `def`, listes en profondeur, boucles `for` / `while`, fichiers `open`, `pip`, classes, Jupyter. Si on te pose la question : « demain data, ce soir on veut un script qui parle ».

### Matériel à confirmer dès l'ouverture

- Un ordinateur (téléphone seul = trop juste pour coder).
- Le dossier `vitrine` du jour 1, si possible.
- Python installé, **ou** <https://www.online-python.com/>.
- Un éditeur : VS Code, Bloc-notes, ou l'éditeur du site en ligne.

**PLAN B**

Si `python` n'est pas reconnu : envoie-les sur <https://www.online-python.com/>. Même exercice, zéro install. À la fin ils copient le code dans `bio.py` pour MyHub.

## DÉROULÉ

## Les 120 minutes

| TEMPS       | BLOC                                  | TU FAIS                                         |
|-------------|---------------------------------------|-------------------------------------------------|
| 0:00 à 0:08 | Accueil                               | Règles, objectif, matériel, promesse du soir.   |
| 0:08 à 0:42 | <code>print</code> , variables, types | Cours + démo + 3 exercices courts.              |
| 0:42 à 0:47 | Pause                                 | 5 minutes. Ils restent connectés.               |
| 0:47 à 1:22 | <code>input</code> et <code>if</code> | Live coding ensemble + 3 exercices.             |
| 1:22 à 1:52 | Mini-projet bio                       | Chacun son <code>bio.py</code> .                |
| 1:52 à 2:00 | Bilan                                 | 3 points clés, dépôt MyHub, teaser jour 3 data. |

Si tu prends du retard : coupe l'exercice 2.3, pas le mini-projet. Le script qui affiche le texte de la vitrine est le souvenir du soir.

### **Ce qu'ils savent faire à 14 h GMT**

1. Écrire un fichier `.py` , lancer `print` , ranger une valeur dans une variable.
2. Demander une réponse avec `input` et brancher un `if / else` .
3. Produire le texte de leur vitrine (prénom, phrase, 3 envies) et le déposer sur MyHub.

## Les huit premières minutes

### À DIRE

« Bienvenue. C'est le soir 2, Python. Hier ta vitrine existait : une page, un lien WhatsApp. Aujourd'hui, un script va écrire le texte de cette page. Même projet, autre métier. Data, demain, ce n'est pas moi : un invité. Vendredi, IA, je reviens. »

### À MONTRER

Écran partagé : `bootcamp.html` (programme du jour 2). Puis un fichier vide `bio.py`, ou l'éditeur d'online-python.com. Annonce le livrable : « à la fin tu as un fichier `bio.py` qui demande ton prénom et affiche le texte de ta vitrine ».

### Quatre règles, comme hier

1. Micro coupé, sauf quand tu parles. Le chat reste ouvert.
2. Questions dans le chat. Tu t'arrêtes toutes les 15 minutes.
3. Jamais trop tard. Ils n'ont jamais programmé ? C'est exactement le public.
4. Un ordinateur. Le téléphone seul ne suffit pas.

### PROMESSE DU SOIR · À RÉPÉTER

« Ce soir tu parles à l'ordinateur en Python, et un script écrit le texte de ta vitrine. »

## Un programme, c'est une recette

### À DIRE

HTML = les murs de la page. Python = la langue des instructions. La recette n'est pas le plat. L'interpréteur, c'est le cuisinier qui lit ligne par ligne. S'il bute, il s'arrête et dit pourquoi.

### À MONTRER · 2 MIN

Tape et lance, lentement :

```
print("Bonjour")  
print("Je m'appelle Awa")
```

Puis casse volontairement : `print(Bonjour)` sans guillemets. Lis le `NameError` à voix haute : « il cherche une variable Bonjour ».

## Variables

### À DIRE

Une variable, c'est une boîte avec une étiquette. Le `=` range, il n'est pas « égal » comme en maths.

`print("Bonjour", prenom)` : Python va chercher dans la boîte.

### À MONTRER

```
prenom = "Awa"  
print(prenom)  
print("Bonjour", prenom)
```

Change `prenom` une fois, relance : toute la suite suit. « C'est pour ça qu'on nomme : on change à un seul endroit. »

## Types · 4 minutes, pas un cours université

### Texte · str

Guillemets. `"20" + "1"` → `"201"`.

### Entier · int

Sans guillemets. `20 + 1` → `21`.

#### À MONTRER

```
print("20" + "1")  
print(20 + 1)  
print(int("20") + 1)
```

Phrase clé : « les guillemets changent le métier de la valeur ».

#### NOMS DE VARIABLES

Pas d'accent ( `prenom` pas `prénom` ), pas d'espace ( `envie_1` ), ne commence pas par un chiffre. S'ils demandent les listes : « pas ce soir, trois variables suffisent ».

## Trois exercices print / variables

### PENDANT QU'ILS CHERCHENT

Ne projette pas les corrigés. Circule dans le chat. Aide d'abord ceux qui n'ont pas encore un `print` qui s'affiche.

#### 1.1 · 4 MIN · QU'EST-CE QUI S'AFFICHE ?

```
ville = "Lomé"  
print("J'habite à")  
print(ville)
```

#### CORRIGÉ 1.1

```
J'habite à  
Lomé
```

Deux lignes. `ville` sans guillemets → contenu de la boîte.

#### 1.2 · 4 MIN · RÉPARE

```
print(Bonjour)  
prenom = Awa  
print prenom  
1ville = "Lomé"
```

#### CORRIGÉ 1.2

1. `print("Bonjour")` — texte = guillemets.
2. `prenom = "Awa"` — idem.
3. `print(prenom)` — parenthèses obligatoires.
4. `ville = "Lomé"` — un nom ne commence pas par un chiffre.

#### 1.3 · 4 MIN · CARTE D'IDENTITÉ

Variables `prenom` et `ville`, deux `print`.

### CORRIGÉ 1.3 · EXEMPLE

```
prenom = "Koffi"  
ville = "Lomé"  
print("Je m'appelle", prenom)  
print("J'habite à", ville)
```

## Cinq minutes, on reste connectés

### À DIRE

« Pause. Ne quittez pas la visio. Ensuite : l'ordinateur écoute, puis il décide. »

Reviens pile à l'heure. Premier écran au retour : `input` .

## input, puis if

### À DIRE

Jusqu'ici le prénom était collé dans le code. `input` , c'est le micro : le programme attend. Toujours un texte, même si tu tapes 20. Pour calculer un âge : `int(...)` .

### À MONTRER · TAPE AVEC EUX

```
prenom = input("Ton prénom : ")
print("Bonjour", prenom)
```

Laisse le curseur clignoter 3 secondes : « il attend. Entrée pour valider. »

## if : le feu tricolore

### À DIRE

Vert tu avances, rouge tu t'arrêtes. `=` range, `==` pose la question. L'indentation, c'est le bloc : ce qui est en retrait appartient au si.

### À MONTRER

```
age = int(input("Ton âge : "))
if age >= 18:
    print("Tu peux voter.")
else:
    print("Pas encore.")
```

Lance une fois avec 16, une fois avec 20. Puis enlève l'indentation sous le `if` : lis

`IndentationError` à voix haute.

### ELIF, 2 MINUTES

Montre `oui` / `non` / autre chose. Ne construis pas un menu à 8 branches. Une décision claire suffit.

## Trois exercices input / if

### 2.1 · 5 MIN · ON TAPE 16

```
age = int(input("Âge : "))
if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

#### CORRIGÉ 2.1

Affiche `Mineur` . 16 n'est pas `>= 18`. Branche `else` .

### 2.2 · 5 MIN · RÉPARE

```
ville = input("Ville : ")
if ville = "Lomé"
print("On est au Togo.")
```

#### CORRIGÉ 2.2

1. `=` → `==` (question, pas rangement).
2. Deux-points après la condition : `if ville == "Lomé":`
3. Le `print` en retrait sous le `if` .

```
ville = input("Ville : ")
if ville == "Lomé":
    print("On est au Togo.")
```

### 2.3 · 6 MIN · ACCUEIL QUI DÉCIDE

Si le prénom tapé est le leur → « C'est moi ! ». Sinon → « Enchanté, ... ».

#### CORRIGÉ 2.3 · EXEMPLE

```
prenom = input("Prénom : ")
if prenom == "Awa":
    print("C'est moi !")
else:
    print("Enchanté,", prenom)
```

Rappelle : la casse compte. `awa` n'est pas `Awa` . Si quelqu'un bloque là-dessus : « ce soir on compare tel quel ».

## bio.py écrit le texte de la vitrine

### À DIRE

Plus de cours. Fichier `vitrine/bio.py`. Il demande, il décide si le prénom est vide, il affiche un texte prêt à coller dans `index.html`. Dépôt MyHub à la fin.

### Cahier des charges · non négociable

- ☐ Fichier nommé `bio.py`.
- ☐ `input` pour le prénom, la phrase, 3 envies.
- ☐ Si prénom vide → « Il me faut un prénom. »
- ☐ Sinon, affichage propre : prénom, phrase, liste des 3 envies.

### SQUELETTE À TAPER EN LIVE · PUIS ILS ADAPTENT

```
prenom = input("Ton prénom : ")

if prenom == "":
    print("Il me faut un prénom.")
else:
    phrase = input("Une phrase sur toi : ")
    envie_1 = input("Envie 1 : ")
    envie_2 = input("Envie 2 : ")
    envie_3 = input("Envie 3 : ")

    print()
    print("--- Texte pour ta vitrine ---")
    print(prenom)
    print(phrase)
    print("Cette semaine je veux :")
    print("-", envie_1)
    print("-", envie_2)
    print("-", envie_3)
```

### CHECKPOINTS TOUTES LES 8 MINUTES

1) Qui a un fichier `.py` qui se lance ? 2) Qui a le `input` prénom ? 3) Qui voit les 3 envies à l'écran ? Aide d'abord ceux sans fichier. Recoller dans le HTML est un bonus, pas un critère de validation.

### BLOCAGES

`bio.py.txt` → extensions Windows. `python` introuvable → [online-python.com](https://online-python.com). Curseur qui clignote → il attend l'`input`. Accents bizarres → UTF-8 ou éditeur en ligne.

## Trois phrases, pas un résumé

### À DIRE

1. « Un programme, c'est une recette. Python la lit dans l'ordre. `print` parle, une variable range. »
2. « `input` écoute, toujours en texte. `if` décide. L'indentation marque le bloc. »
3. « Ton `bio.py` écrit le texte de la vitrine. Même page qu'hier, autre métier. »

### Avant de quitter la visio

1. Garder le dossier `vitrine` : `index.html` et `bio.py`.
2. Lancer le script une dernière fois.
3. Déposer `bio.py` sur MyHub, espace étudiant, devoir du jour 2.
4. RDV jour 3 : demain, même heure, même visio. Data, un invité. Ordinateur encore obligatoire.

### CLÔTURE

« Demain : data. Trois chiffres collés sur ta page. Toi tu reviens vendredi pour l'IA. Message groupe après l'appel. »

### POUR TOI, APRÈS L'APPEL

Félicite le script qui s'exécute, pas le plus joli `print`. Message groupe : lien espace étudiant + rappel du fichier `bio.py`. Tu n'as pas à tout savoir de Python. Tu as à faire vivre 120 minutes claires, un fichier qui se lance, un texte de vitrine à l'écran.